through its package manager, Cargo.

- **Safety-critical systems:** Rust is being used in a number of safety-critical systems, such as the Firefox web browser and the Oxide browser engine, due to its focus on memory safety and concurrency.
- **Easy to learn:** Although it's debatable, in my opinion, Rust has a friendly and approachable syntax, making it easy for new users to pick up and learn.

Ezhil

In the past, there have been several efforts to create programming languages that incorporate Indian languages. Ezhil is one such programming language. It is a cutting-edge, open source, interpreted programming language that was specifically designed for native Tamil speakers, particularly for students. Its goal is to simplify the process of learning programming and numeracy for Tamil speakers by incorporating Tamil keywords and grammar, while also providing logical constructs that are similar to those found in English-based programming languages. As the first freely available programming language in Tamil, Ezhil was officially announced in 2009 after being developed since 2007. The language is intended to make computer programming and numeracy more accessible for individuals who may not be fluent in English.

I am a native speaker of Malayalam, and I noticed the lack of a programming language like Ezhil that is tailored to the Malayalam language. This led me to the idea of creating a language that would allow us to write computer programs in Malayalam. Because of the benefits discussed earlier, I opted to utilise Rust for the development of my new language, Malluscript.

Malluscript

As people from the Indian state of Kerala primarily speak Malayalam, they are often referred to as 'Mallus' across India. Malluscript felt like an appropriate choice of name for a programming language for Mallus that enables them to write programs in Malayalam. One of the main aims of this language was to use it in an instant coding competition, in which students would be provided with a brand new language to solve certain problems in. Since the main objective was to attract younger students for a competition, I designed Malluscript in a comic tone, involving trendy memetic keywords as tokens.

Like any standard programming language, Malluscript also contains a lexer, a parser and an executor. This esoteric scripting language follows the same structure and principles. The idea was to make it easy to understand and accessible for younger students while still adhering to the fundamental concepts of programming.

Writing a lexer

The lexer's main job is to identify the individual tokens in the input stream and to classify them based on their type. For example, a lexer might identify a sequence of characters like 'if' as a keyword token, or a sequence of digits as a numerical value token. Malluscript's lexer does the same; it converts the text representation of language to an iterator of token, which will be used by the parser in the next stage. Figure 1 shows the list of tokens in Malluscript.

Malluscript has 31 different tokens, with some of the keywords having multiple aliases. When I was writing Malluscript initially, I only intended it to support Manglish (Malayalam language written using English words) since that is the most common way Malayalam speaking people communicate through the internet. But after the first release, I decided to incorporate actual Malayalam Unicodes too. Therefore, Malluscript's keywords can have multiple aliases in both English as well as Malayalam keywords. I wrote a specialised function in the lexer to detect this as well. Figure 2 shows the keyword mapping in Malluscript.

Lexer then reads through the input (which is our program) and converts it to an iterator of tokens, which will be used by parser.

```
impl std::fmt::Display for TokenType {
    fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {
        match self {
            TokenType::Declaration => write!(f, "pwoli_sadhanam"),
            TokenType::Write => write!(f, "dhe_pidicho"),
            TokenType::InputString => write!(f, "address_thada"),
            TokenType::InputNumber => write!(f, "number_thada"),
            TokenType::LeftBrace => write!(f, "{{"),
            TokenType::RightBrace => write!(f, "}}"),
            TokenType::If => write!(f, "seriyano_mwone"),
            TokenType::Else => write!(f, "seri_allel"),
            TokenType::Loop => write!(f, "repeat_adi"),
            TokenType::Assignment => write!(f, "="),
            TokenType::Plus => write!(f, "+"),
            TokenType::Minus => write!(f, "-"),
            TokenType::Product => write!(f, "*"),
            TokenType::Divide => write!(f, "/"),
            TokenType::Modulo => write!(f, "%"),
            TokenType::GreaterThan => write!(f, "veluthane"),
            TokenType::LessThan => write!(f, "cheruthane"),
            TokenType::EqualTo => write!(f, "same_aane"),
            TokenType::NotEqual => write!(f, "same_alle"),
            TokenType::SemiColon => write!(f, ";"),
            TokenType::OpenParantheses => write!(f, "("),
            TokenType::CloseParantheses => write!(f, ")"),
            TokenType::Um => write!(f, "um"),
            TokenType::Nekal => write!(f, "ne_kal"),
            TokenType::Literal(literal) => write!(f, "{}", literal),
            TokenType::Integer(number) => write!(f, "{}", number),
            TokenType::Float(number) => write!(f, "{}", number),
            TokenType::Symbol(symbol) => write!(f, "{}", symbol),
            TokenType::Comma => write!(f, ","),
            TokenType::Return => write!(f, "return"),
            TokenType::AngleOpen => write!(f, "<"),
            TokenType::AngleClose => write!(f, ">"),
        }
    } fn fmt
} impl Display for TokenType
```

Figure 1: List of tokens in Malluscript